Doubly Linked List – Insert At End –Space Complexity and Algorithm.

Program **:**

```
void insertAtEnd(int data)
{
    Node *newNode = (Node *)malloc(sizeof(Node));
    newNode->data = data;
    newNode->next = nullptr;

    if (head == nullptr)
    {
        newNode->prev = nullptr;
        head = newNode;
        cout << data << " inserted at the end." << endl;
        return;
    }
    Node *temp = head;
    while (temp->next != nullptr)
    {
        temp = temp->next;
    }
    temp->next = newNode;
    newNode->prev = temp;
    cout << data << " inserted at the end." << endl;
}

…
        case 2:
            cout << "Enter data to insert: ";
            cin >> data;
            insertAtEnd(data);
            break;
```

Space Complexity

Auxiliary Space

Auxiliary space is the extra memory used by the function, not including the space taken by the inputs.

In the `insertAtEnd` function, the new memory usage is:

1. `Node *newNode`: One new node is allocated. The size of this node (`sizeof(Node)`) is **constant**.
2. `Node *temp`: One temporary pointer is created to traverse the list. The size of a pointer is also **constant**.

Since the amount of extra memory used does not change with the size of the linked list, the auxiliary space complexity is O(1).

- **Auxiliary Space: O(1)**

Input Space

Input space is the memory used to store the inputs. We can analyze this from two perspectives.

1. Parameter-only Definition

This view considers only the space taken by the function's direct parameters. The function is `void insertAtEnd(int data)`.

- The input is a single integer, `data`, which occupies a fixed, constant amount of memory.

Under this definition, the input space is constant.

- **Input Space (Parameter-only): O(1)**

2. Full-Data Definition

This broader view includes the space taken by the entire data structure the function operates on—the doubly linked list itself.

- If the linked list contains **n** nodes, the total memory it occupies is proportional to **n**.

Under this definition, the input space depends on the size of the list.

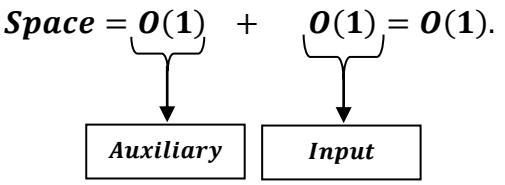
- **Input Space (Full-Data): O(n)**

Total Space Complexity

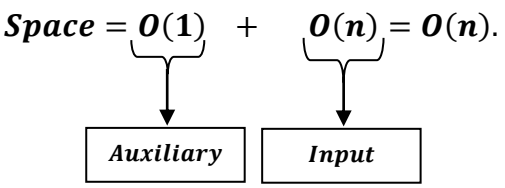
Total Space Complexity = Auxiliary Space + Input Space

Based on the definition of input space you use, the total space complexity changes.

- **Common Interpretation (Parameter-only input):**



- **Holistic Interpretation (Full-data input):**



  In most academic and interview contexts, when asked for the space complexity of a function, the **auxiliary space complexity (O(1))** is the expected answer.

The traversal loop (`while (temp->next != nullptr)`) affects the function's time complexity (O(n)) but not its space complexity.

Algorithm: Insert at End (Doubly Linked List)

$DINSEND(START, ITEM)$
$\left((Node*)malloc(sizeof(Node))\ is\ availability\ of\ space\ for\ insertion\ of\ Data,\ hence\ ,lets\ name\ it\ AVAIL.\right)$

1. [$ALLOCATE\ NEW\ NODE$]

   $Set\ NEW := AVAIL.$

2. [$COPY\ DATA$ & $SET\ FORWARD\ POINTER$]

   $a. Set\ INFO[NEW] := ITEM.$

   $b. Set\ NEXTLINK[NEW] := NULL$ [$Sets\ the\ forward\ link\ of\ the\ new\ node\ to\ NULL, as\ it\ will\ be\ the\ last\ node$].

3. [$CHECK\ IF\ LIST\ IS\ EMPTY$]

   $If\ START = NULL, then$:

   $a. Set\ PREVLINK[NEW] := NULL.$

   $b. Set\ START := NEW.$

   $c. Write: ITEM$ "$inserted\ at\ the\ end.$" $and\ EXIT.$

   [$END\ OF\ IF\ STRUCTURE$]

   [$List\ is\ not\ empty$?]:

   $a. Set\ PTR := START$ [$PTR\ is\ a\ temporary\ pointer\ assigned\ to\ the\ start\ of\ the\ list$].

   $b. Repeat\ while\ NEXTLINK[PTR] \neq NULL$: [$Traverses\ the\ list\ to\ find\ the\ last\ node$].

   $Set\ PTR := NEXTLINK[PTR]$ [$Updates\ PTR\ to\ point\ to\ the\ next\ node$].

   [$END\ of\ Loop$]

   $c. Set\ NEXTLINK[PTR] := NEW$ [$Sets\ the\ NEXT\ pointer\ of\ the\ last\ node\ (PTR)\ to\ the\ new\ node$].

   $d. Set\ PREVLINK[NEW] := PTR$ [$Sets\ the\ PREV\ pointer\ of\ the\ new\ node\ back\ to\ the\ former\ last\ node\ (PTR)$].

   $e. Write: ITEM$,"$inserted\ at\ the\ end.$"

4. $Exit$

[$END\ OF\ ALOGITHM$]